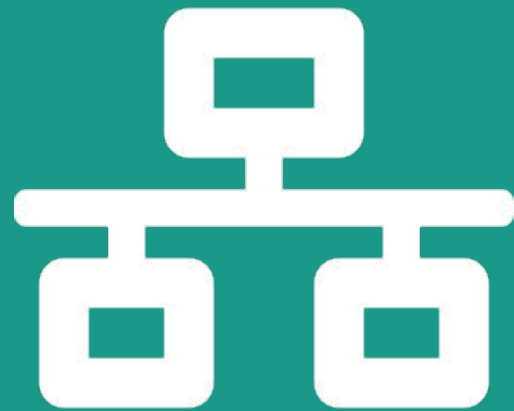


The Map, the Model & How Systems Talk

Day 1 (AM) - Session 1 - Web API Development



Session 1 - Learning Outcomes



From a blank canvas to a working data model

LO1

Describe the Tuk-Tuk Monitoring System: its IoT architecture, geographic hierarchy, and the two distinct API consumer roles.

LO2

Explain how two systems communicate: client, server, a shared language, and an agreed protocol - applied to our system.

LO3

Apply ER notation to model the domain and explain why the data model precedes every other design decision (REST API Design §3).

A

The Business Case

Understanding the problem before designing the solution



The Tracking Problem



- Sri Lanka Police need to monitor thousands of civilian tuk-tuks moving across provinces, districts, and stations.
- Each tuk-tuk carries an IoT GPS device that transmits its position automatically - no driver action required.
- Emergency operators, Traffic OICs, and field officers can access this information in real time and review historical records as needed.
- The API sits in the middle: IoT devices write to it, police applications read from it.

ANALOGY

A tuk-tuk meter logs every fare automatically - no driver action. A central licensing database is the API; transport inspectors query it later. Limit: a meter records fare amounts, not GPS coordinates. Same producer-consumer pattern; different data type.

The System at Scale



9

Provinces

Top-level jurisdiction

25

Districts

Mid-level filtering

200+

Tuk-Tuks

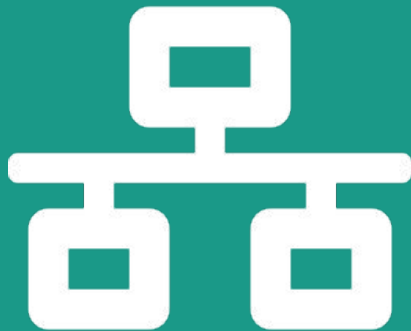
IoT data producers

Element	Seed Scale	Role in the System
Provinces	9	Top-level jurisdiction scope
Districts	25	Mid-level read scoping
Police Stations	20+	Station-level access boundary
Vehicles	200+	IoT data producers; each has device_id
Location Pings	1 week+	Append-only time-series history

B

How Two Systems Talk

Client, server, shared language, and an agreed protocol



Client and Server - Two Roles, One Conversation



- Every networked system divides into two roles: one asks, one answers.
- The client makes a request. The server processes it and returns a response.
- The client does not need to know how the server works internally - only how to ask correctly.
- One server serves many clients simultaneously. One client can talk to many servers.

ANALOGY

A passenger at a trishaw stand says "Pettah" - they see no driver roster, no assignment process; they get one answer. The internal routing stays completely hidden. Limit: a dispatcher remembers regular customers; an HTTP server does not - each request is treated as the first.

A Shared Language - What Both Sides Agree On



- Client and server must agree on the shape of what is exchanged - a format both sides can read and write.
- This shared format is separate from the protocol (how data travels) and from the data model (what the data represents).
- Examples: JSON, XML, plain text, CSV. Each is a different shared language for the same underlying data.
- Choosing a format before you understand your data model locks you in too early. We defer this choice to Session 5.

REST API Design principle: specify the data model first, choose the representation format later. Today we model. Session 5 is where we choose JSON - and we will understand exactly why.

A Protocol - The Rules of the Road



- A protocol is a complete, agreed set of rules so that a conversation between two systems works reliably.
- It defines: how to open the conversation, how to frame a request, how to frame a reply, how to signal failure, how to close.
- Without an agreed protocol the server cannot tell where one message ends and the next begins.
- Protocols exist at every network layer - physical, transport, application. The web uses HTTP at the application layer.

ANALOGY

A bus pass check works because both sides agreed on the steps in advance - you hold it out, they nod or question. No one improvises on the spot. Limit: a conductor can bend the rules for a child without a pass; HTTP cannot - every rule is machine-enforced with no flexibility or mercy.

HTTP - The Protocol the Web Runs On



- HTTP (HyperText Transfer Protocol) is the application-level protocol of the entire web.
- Every API call in this module travels over HTTP - or its encrypted version, HTTPS.
- HTTP defines verbs for asking: GET (read), POST (create), PUT (replace), DELETE (remove).
- HTTP defines status codes for answering: 200 OK, 201 Created, 404 Not Found, 401 Unauthorised...
- Critical property: HTTP is stateless - the server treats every request as entirely independent.

Forward pointer - verbs and status codes are taught in full in Session 7 (Day 4 AM). Today: HTTP is our agreed protocol. Knowing it exists and that it is stateless is enough.



The Tuk-Tuk System

Applying the concepts: two client types, one API



Two Clients, Two Roles, One API

Tracking Devices (Write)

- IoT GPS units fitted to each civilian tuk-tuk
- Authenticate as their specific vehicle
- POST location pings only - fully autonomous, no human involvement
- Can only write data for their own vehicle - never another's

Police Users (Read)

- HQ, provincial, district, and station officers
- GET location data and movement history - read only
- Scoped by jurisdiction - a station officer reads only their station's vehicles
- Never POST data. The read path and write path never cross.

The write path and the read path are separate by design - different authentication, different permissions, different data flows. This separation shapes every security decision in the system.



THE WRITE-READ SPLIT

The device writes.
The police read.
The API must enforce that line.

Every authentication decision, every permission check, every scope in this system flows from this one fact.

D

The Data Model (REST API Design §3)

Model first. Format later. URLs last.



Why the Data Model Comes First



- **Step 1: model the data before writing a single URL or choosing a format.**
- **The data model is implementation-independent - it describes what exists, not how it is stored or transferred.**
- **Resources, URIs, representations, and HTTP methods all derive from this model. Get the model wrong; everything above it is wrong.**

ANALOGY

An architect commits to a floor plan before choosing the tiles - moving walls on paper costs nothing; after construction it costs everything. The data model is that floor plan; URLs, fields, and status codes are the interior finish. Limit: unlike a floor plan, a live public API cannot be quietly revised - every client breaks.

Boundary: no URLs, no JSON, no XML today. We are modelling. URIs: S3. Format: S5.

ER Notation - A Quick Refresher

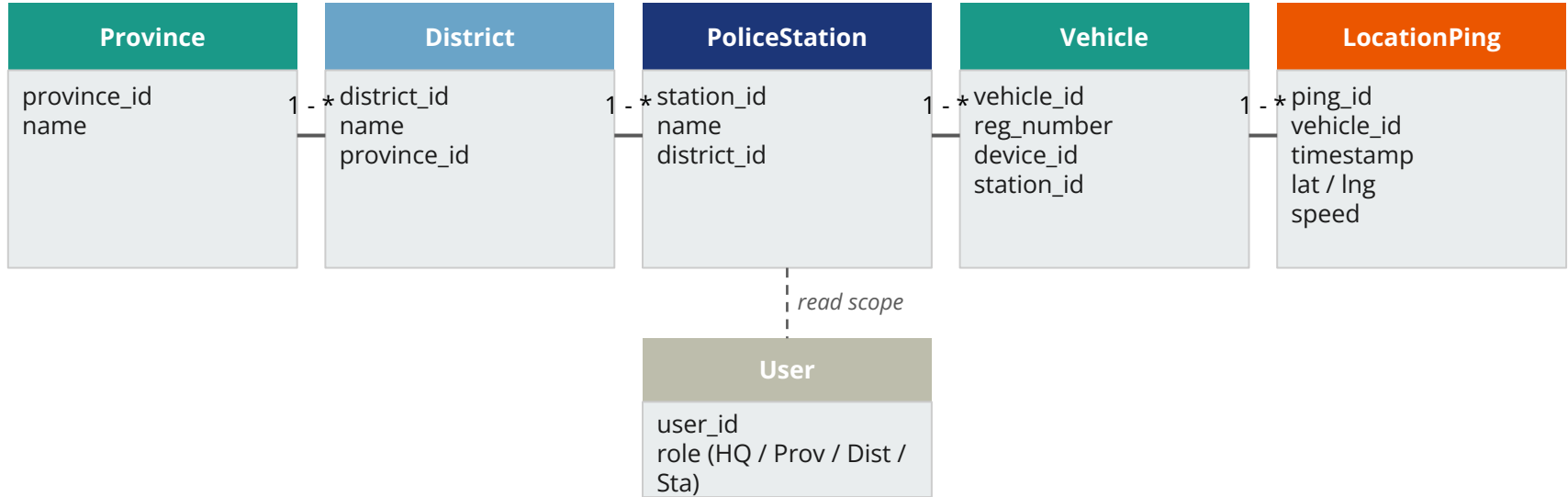


- An entity is a thing that exists independently and has attributes (e.g. Vehicle with a registration number and a device_id).
- A relationship connects two entities (e.g. a Vehicle is registered at a PoliceStation).
- Cardinality tells you how many: one station has many vehicles (1-to-many); one ping belongs to one vehicle (many-to-1).
- We are modelling the domain - not SQL tables. No primary-key syntax, no foreign-key constraints yet.

ANALOGY

Grama Niladhari knows every household in its area; the DS office covers multiple GN areas; the district covers multiple DS offices; the province covers multiple districts. This IS our Province - District - Station hierarchy. You use this structure daily. Limit: GN carries legal authority and jurisdiction; our model carries read-scope only.

The Domain Model



One Design Decision Worth Naming



- `device_id` is an attribute of `Vehicle` - not a separate `Device` entity.
- Every tuk-tuk has exactly one GPS device; the device exists only to serve the vehicle it is fitted to.
- A separate `Device` entity would add a join, a concept, and a table with no analytical value at this scope.
- This is a deliberate simplification - a valid engineering choice for the problem as defined.
- At the viva, you will be asked to explain this decision. "The skill file told me to" is not an answer.

Design note: if the system later needed to track device replacements, calibration history, or battery status, a separate `Device` entity would earn its place. For this coursework scope, it does not. Know when simplification is correct, not just convenient.

LocationPing - Its Own Entity, Not a Field



- A vehicle accumulates hundreds of pings per day - each with a timestamp, latitude, longitude, and speed.
- This is a time-series: append-only, never updated, continuously growing. It cannot be one field on Vehicle.
- Keeping it as a separate entity is what later makes filtering by time window, pagination, and historical queries arise naturally from the design.

ANALOGY

An electricity meter appends a new reading each billing cycle - it never overwrites a single "current reading" field. The utility company queries the history of readings, not a single value. Limit: a meter is read monthly; a GPS ping arrives every 30 seconds. Same append-only pattern; very different data volume.

What a Code Generator Gets Wrong Here



- Invents a separate Device entity as the primary data source - obscuring the Vehicle relationship entirely.
- Bakes format names into the model: json_payload, response_data - conflating representation with structure.
- Adds last_known_lat and last_known_lng as fields on Vehicle - collapsing the time-series into a single value.
- Uses SQL-style CamelCase field names - pre-empting implementation decisions the data model must not make.
- Makes the police User entity the data producer - conflating who reads the API with the device that writes to it.

Every one of these mistakes appears in a typical first-pass generated ER. Your job today - and at the viva - is to spot each mistake and explain exactly which design principle it violates.

From Data Model to Resources - A Preview

1

Atomic

A single instance (e.g. one Vehicle, one Province)

2

Collection

A set of atomics (e.g. all Vehicles in a district)

3

Composite

A bundle as one (Vehicle + its last ping)

4

Processing Function

A derived computation (last known position)

5

Controller

A multi-resource operation (rare in our system)

Full derivation of resources from the domain model: Session 3 (Day 2 AM). Today, know the five kinds exist.

Demo - Resource Type Classifier: drop an entity from the domain model and immediately see which of the five kinds it resolves to, and the rule that decided it. Open the demo now.

Clients Win Over Data

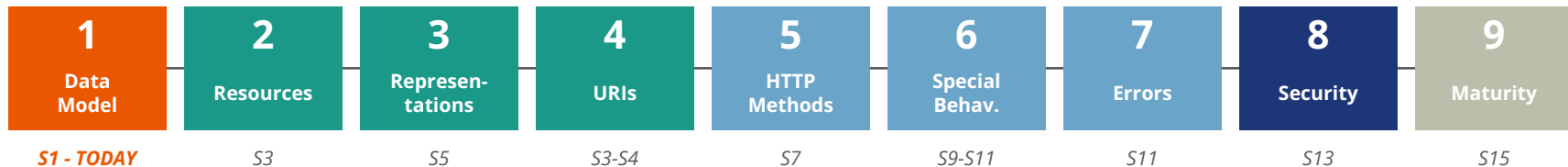


- The resource model is not a direct copy of the data model (REST API Design §4 principle).
- The domain model drives implementation; the resource model is driven by how clients actually need to access data.
- Police do not want "all location pings ever." They want "pings for WP-1234 in the last 24 hours, Western Province."

ANALOGY

A library catalogue holds every book, but readers ask "all books by this author at Colombo branch, published after 2019" - never "show me all books." The catalogue is the data model; the search interface is the resource model, shaped entirely by what readers need. Limit: our API scopes reads to jurisdiction - not every combination a reader might want is valid.

Your Map for the Next 8 Days



Step 1 is where we are right now. The complete map is visible from Day 1 so you always know where you are in the design process and what comes next.



Discussion

Look at these five paths:

/getVehicleLocation /vehicles/WP-1234/location /updatePing /pings /calculateRoute

Which feel like things? Which feel like actions? Why does that difference matter for an API design?

5 minutes - discuss with the person next to you, then share one observation with the room.

This question returns in Session 3 when we write the real URI naming rules (REST API Design §5.1).

Activity

ER Model the Case

1. Open draw.io, paper, or your notes app.
2. Draw the 5 entities: Province, District, PoliceStation, Vehicle, LocationPing.
3. Add at least 3 key attributes per entity.
4. Draw relationships and mark cardinality (1-to-many).
5. Add the User entity, its role attribute, and its jurisdiction link.
6. Compare with the reference model on screen. Note every difference.
7. Be ready to explain one design decision at the micro-viva.

25 min

individual + share

Tools

diagrams.net (draw.io)
Paper + pen
Google Slides
Any diagram tool

Questions & Discussion

Session 1 complete. Session 2 (PM, this afternoon) starts with seed data generation and your first public deployment.



Disclaimer & Copyrights



The information presented in this presentation is intended for general informational purposes only. It is not meant to be comprehensive or to constitute professional advice. Any reliance you place on such information is strictly at your own risk. While every effort is made to keep the information accurate and up-to-date, no representations or warranties of any kind are made about its completeness, accuracy, reliability, or suitability.

This presentation may include references to third-party content. I do not endorse external sites and am not responsible for their content.

Any action taken upon the information presented is strictly at your own discretion. I will not be liable for any losses or damages in connection with its use.

This disclaimer is subject to change without notice.

@ Images and artworks used in this presentation are for general informational purposes only. It may contain copyrighted material, the use of which may not have been specifically authorised by the copyright owner. This material is available in an effort to explain issues relevant to the topic. This should constitute a 'fair use' of any such copyrighted material.

Last modified: 13 Jun 2026